

Data Structures & Algorithms

Core Theoretical Concepts, Practical Examples, and 25 Multiple-Choice Practice Questions with Answer Key

Part 1: Core Data Structures Theory & Examples

1. Arrays

Theory: An array is a linear data structure consisting of a collection of elements, each identified by at least one array index or key. Elements are stored in contiguous memory locations, allowing for constant-time $O(1)$ random access by index.

- **Time Complexity:** Access: $O(1)$ | Search: $O(n)$ | Insertion/Deletion: $O(n)$
- **Use Cases:** Storing sequential data, implementing dynamic tables, matrix representations.

PYTHON EXAMPLE: ARRAY OPERATIONS

```
# Dynamic Array implementation in Python
arr = [10, 20, 30, 40, 50]

# Access element at index 2 (O(1))
print("Element at index 2:", arr[2]) # Output: 30

# Insert element at specific index (O(n))
arr.insert(2, 25) # [10, 20, 25, 30, 40, 50]

# Append element at end (Amortized O(1))
arr.append(60)
```

2. Linked Lists

Theory: A Linked List is a linear data structure where elements (called nodes) are not stored in contiguous memory. Each node contains a data field and a reference (link) to the next node in the sequence. Types include Singly Linked List, Doubly Linked List, and Circular Linked List.

- **Time Complexity:** Access: $O(n)$ | Insertion at Head: $O(1)$ | Deletion at Head: $O(1)$
- **Use Cases:** Dynamic memory allocation, undo functionality, implementing Stacks and Queues.

PYTHON EXAMPLE: SINGLY LINKED LIST NODE

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert_at_head(self, data):
```

```

        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

# Creating a list: 10 -> 20 -> None
ll = LinkedList()
ll.insert_at_head(20)
ll.insert_at_head(10)

```

3. Stack (LIFO)

Theory: A Stack is a linear data structure following the **Last In, First Out (LIFO)** principle. Elements are added and removed from the same end, called the "top". Key operations are `push()` (insert), `pop()` (remove), and `peek()` / `top()`.

- **Time Complexity:** Push: $O(1)$ | Pop: $O(1)$ | Top: $O(1)$
- **Use Cases:** Function call stacks (recursion), expression evaluation, back-button in web browsers.

PYTHON EXAMPLE: STACK IMPLEMENTATION

```

stack = []

# Push elements onto stack
stack.append('A')
stack.append('B')
stack.append('C')

# Top element
print("Top element:", stack[-1]) # Output: C

# Pop element
popped = stack.pop() # Removes 'C'
print("Popped:", popped)

```

4. Queue (FIFO)

Theory: A Queue is a linear data structure that operates on the **First In, First Out (FIFO)** principle. Elements enter at the rear (enqueue) and exit from the front (dequeue).

- **Time Complexity:** Enqueue: $O(1)$ | Dequeue: $O(1)$
- **Use Cases:** CPU scheduling, printer task queues, Breadth-First Search (BFS) in graphs.

PYTHON EXAMPLE: QUEUE USING COLLECTIONS.DEQUE

```

from collections import deque

queue = deque()

# Enqueue (Enqueue at rear)
queue.append("Task 1")
queue.append("Task 2")

```

```
# Dequeue (Dequeue from front)
first_task = queue.popleft() # Output: "Task 1"
```

5. Binary Search Tree (BST)

Theory: A Binary Search Tree is a hierarchical non-linear data structure where each node has at most two children. The left subtree contains nodes with values less than the parent, and the right subtree contains values greater than the parent.

- **Time Complexity:** Search / Insert / Delete: Average $O(\log n)$, Worst Case $O(n)$ (unbalanced).
- **Use Cases:** Fast lookup tables, dynamic sorting, range queries.

PYTHON EXAMPLE: BST INSERTION & TRAVERSAL

```
class BSTNode:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

def insert(root, key):
    if root is None:
        return BSTNode(key)
    if key < root.key:
        root.left = insert(root.left, key)
    else:
        root.right = insert(root.right, key)
    return root
```

6. Hash Table / Hash Map

Theory: A Hash Table maps key-value pairs using a hash function to compute an index into an array of buckets. It provides efficient key-based lookup by converting keys into integer indices.

- **Time Complexity:** Search / Insertion / Deletion: Average $O(1)$, Worst Case $O(n)$ (collisions).
- **Use Cases:** Database indexing, caching, associative arrays/dictionaries.

Summary of Time & Space Complexities

Data Structure	Access (Avg)	Search (Avg)	Insertion (Avg)	Deletion (Avg)	Space Complexity
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Singly Linked List	$O(n)$	$O(n)$	$O(1)$ [at head]	$O(1)$ [at head]	$O(n)$
Stack	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Queue	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Binary Search Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Hash Table	N/A	$O(1)$	$O(1)$	$O(1)$	$O(n)$

Part 2: Practice Questions (25 MCQs)

Select the single correct answer for each of the following multiple-choice questions.

Q1. Which data structure works on the LIFO (Last In, First Out) principle?

- A) Queue
- B) Stack
- C) Array
- D) Linked List

Q2. What is the time complexity of accessing an element in an Array by its index?

- A) $O(1)$
- B) $O(n)$
- C) $O(\log n)$
- D) $O(n^2)$

Q3. Which data structure is best suited for Breadth-First Search (BFS) graph traversal?

- A) Stack
- B) Priority Queue
- C) Queue
- D) Binary Search Tree

Q4. In a Singly Linked List, how many pointer/reference fields does each node contain?

- A) 0
- B) 1
- C) 2
- D) Depends on size

Q5. What is the worst-case time complexity of searching an element in a balanced Binary Search Tree (BST)?

- A) $O(1)$
- B) $O(\log n)$
- C) $O(n)$
- D) $O(n \log n)$

Q6. Which of the following data structures allows dynamic memory allocation?

- A) Static Array
- B) Linked List
- C) Simple Struct
- D) Primitive Variables

Q7. What condition indicates that a linear Queue implemented via dynamic array is empty?

- A) `Front == Rear`
- B) `Front == -1`
- C) `Rear == MAX - 1`
- D) `Front == Rear + 1`

Q8. Which tree traversal technique yields nodes in non-decreasing order for a Binary Search Tree?

- A) Pre-order
- B) Post-order
- C) In-order
- D) Level-order

Q9. What is the main advantage of a Doubly Linked List over a Singly Linked List?

- A) Less memory consumption
- B) Bidirectional traversal
- C) $O(1)$ searching
- D) Random memory access

Q10. Underflow in Stack occurs when attempting to perform which operation?

- A) Push on full stack
- B) Pop from empty stack
- C) Peek on non-empty stack
- D) Traversing stack

Q11. Which technique is used to resolve collisions in Hash Tables?

- A) Chaining
- B) Binary Search
- C) In-order Traversal
- D) Dynamic Programming

Q12. What is the height of a balanced Binary Tree with n nodes?

- A) $O(n)$
- B) $O(\log n)$
- C) $O(1)$
- D) $O(n^2)$

Q13. Which evaluation order is used for Postfix expressions?

- A) Operator between operands
- B) Operator after operands
- C) Operator before operands
- D) No operators used

Q14. In a Max-Heap, where is the largest element located?

- A) Leftmost Leaf
- B) Rightmost Leaf
- C) Root Node
- D) Middle Node

Q15. What is the maximum number of children a node can have in a Binary Tree?

- A) 1
- B) 2
- C) 3
- D) Unlimited

Q16. Which data structure is used by the system for function call overhead and recursion management?

- A) Call Stack
- B) Queue
- C) Heap
- D) B-Tree

Q17. What is the worst-case time complexity of insertion in an unbalanced BST?

- A) $O(1)$
- B) $O(\log n)$
- C) $O(n)$
- D) $O(n \log n)$

Q18. A Circular Queue overcomes which major drawback of a linear array-based Queue?

- A) High search cost
- B) Memory wastage from unused spaces
- C) Slow enqueue speed
- D) Dynamic memory overhead

Q19. Which algorithm is used to find the shortest path in a weighted graph?

- A) Kruskal's
- B) Dijkstra's
- C) Prim's
- D) Depth-First Search

Q20. What is the time complexity to insert a node at the head of a Singly Linked List?

- A) $O(1)$
- B) $O(n)$
- C) $O(\log n)$
- D) $O(n^2)$

Q21. Which of the following is a non-linear data structure?

- A) Array
- B) Stack
- C) Tree
- D) Queue

Q22. What is a Hash Collision?

- A) Two keys produce the same hash index
- B) A key maps to an invalid index
- C) Hash table runs out of memory
- D) Hash function returns negative value

Q23. In Depth-First Search (DFS) of a graph, which data structure is implicitly or explicitly utilized?

- A) Queue
- B) Stack
- C) Heap
- D) Array

Q24. What is the auxiliary space complexity of an in-place array sorting algorithm like HeapSort?

- A) $O(1)$
- B) $O(n)$
- C) $O(\log n)$
- D) $O(n^2)$

Q25. In a Circular Linked List, the last node points to which node?

- A) NULL
- B) Head (First Node)
- C) Itself
- D) Middle Node

Part 3: Answer Key & Quick Explanations

Q.No	Correct Answer	Explanation / Concept Key
Q1	B) Stack	Stacks push and pop from the top, making the last inserted element the first to be retrieved (LIFO).
Q2	A) $O(1)$	Arrays store elements in contiguous memory; address calculation formula yields $O(1)$ direct access.

Q.No	Correct Answer	Explanation / Concept Key
Q3	C) Queue	BFS explores nodes level by level using a FIFO Queue structure.
Q4	B) 1	Singly linked lists store a single pointer reference to the next node in sequence.
Q5	B) $O(\log n)$	Height of a balanced tree with n nodes is $\log n$, bounding search operations to $O(\log n)$.
Q6	B) Linked List	Nodes are dynamically allocated on heap memory during insertion operations.
Q7	B) <code>Front == -1</code>	Standard linear queue initialization sets Front to -1 to denote zero elements.
Q8	C) In-order	In-order traversal (Left, Root, Right) visits BST keys in sorted ascending order.
Q9	B) Bidirectional traversal	Nodes have both 'prev' and 'next' pointers, permitting movement in both directions.
Q10	B) Pop from empty stack	Attempting to delete or retrieve from a stack containing zero elements causes Underflow.
Q11	A) Chaining	Chaining resolves hash collisions by storing colliding elements in a linked list bucket.
Q12	B) $O(\log n)$	Balanced trees maintain height proportional to the binary logarithm of the node count.
Q13	B) Operator after operands	Postfix notation (Reverse Polish) places operators immediately following their operands (e.g., <code>AB+</code>).
Q14	C) Root Node	By Max-Heap property, the root node contains the maximum element among all nodes.
Q15	B) 2	By definition, every node in a binary tree has at most two child nodes (left and right).
Q16	A) Call Stack	System call stack tracks active function calls, local variables, and return memory addresses.
Q17	C) $O(n)$	An unbalanced BST can degenerate into a single linear linked list leading to $O(n)$ operations.
Q18	B) Memory wastage	Circular queues wrap around indices to reuse vacated positions from dequeued items.
Q19	B) Dijkstra's	Dijkstra's algorithm calculates single-source shortest paths in non-negative weighted graphs.
Q20	A) $O(1)$	Updating the head pointer to point to the new node takes constant time $O(1)$.
Q21	C) Tree	Trees organize data hierarchically, unlike linear structures like Arrays, Stacks, and Queues.
Q22	A) Two keys produce same index	A collision occurs when a hash function maps two distinct keys to the identical array index.
Q23	B) Stack	DFS relies on a Stack (or function call stack via recursion) to backtrack to previous vertices.
Q24	A) $O(1)$	In-place heapsort reorganizes array elements directly without allocating additional array memory.
Q25	B) Head (First Node)	In a circular linked list, the final node's next pointer references the first node (Head).